TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

Lab 1: The OS as a Resource Manager

Operating Systems

Student: Gafenco Victor

Group: FAF-241

Instructor: Assistant lecturer, Patricia Reițman

Chișinău - 2026

CONTENTS

Abstract **3**

Abbreviations **4**

Introduction **5**

1 File and directory management **6**

1.1 Directory structure 6

1.2 Creation, copying, renaming, and removal 7

1.3 Ownership and permissions 8

2 Process management **9**

2.1 Listing processes and identifying PID 1 9

2.2 Starting and terminating a background task 10

2.3 Inspecting process state through the kernel 11

3 Memory management **12**

3.1 Physical memory and swap 12

3.2 Resident memory of a sleeping process 12

4 Device and storage management **14**

4.1 Root filesystem and disk usage 14

4.2 Device entries and file-like interfaces 15

Conclusions **16**

ABSTRACT

This laboratory examines how an operating system manages files, processes, memory, and devices using an Ubuntu virtual machine. Terminal commands were used to create and remove files, change access permissions, inspect running processes, terminate a background task, and read kernel information through the process filesystem. Memory and storage commands provided quantitative snapshots of the machine. The file owner and group were both `victor`; the process listing contained approximately 222 processes; memory inspection reported 3.3 GiB of total memory and no configured swap. The root filesystem was mounted from `/dev/sda2` using ext4. Screenshots document each resource-management task. Values from different captures are treated as separate observations because resource usage changes during execution.

Keywords: file permissions, processes, memory, devices, storage, linux

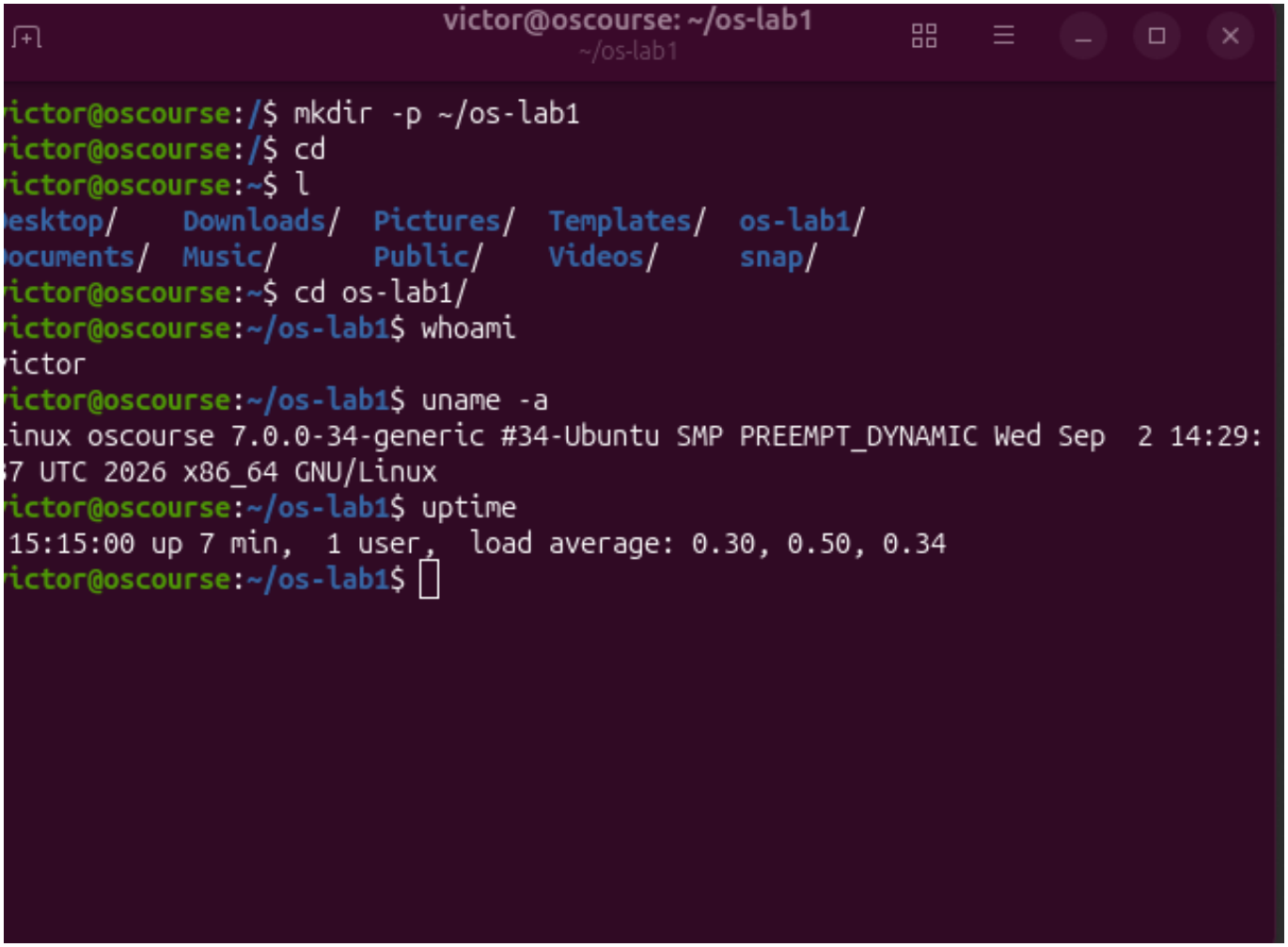
ABBREVIATIONS

OS	operating system
VM	virtual machine
PID	process identifier
RAM	random-access memory
RSS	resident set size
I/O	input/output
GiB, MiB, KiB	binary units: 2^{30}, 2^{20}, and 2^{10} bytes

INTRODUCTION

The operating system mediates between programs and hardware by allocating resources and enforcing access rules. This laboratory follows the four resource-management areas in the course handout: files, processes, memory, and devices [**?**]. The work was performed through the terminal in an Ubuntu VM. The root-directory experiment also uses the openSUSE Tumbleweed host, identified in its command transcript.

The VM uses account `victor`, hostname `oscourse`, and the `7.0.0-34-generic` Linux kernel on x86-64. The initial capture shows seven minutes of uptime. The work directory was `/home/victor/os-lab1`. The screenshot below records the environment. Later measurements were taken in separate sessions and should not be interpreted as simultaneous.



1 FILE AND DIRECTORY MANAGEMENT

The filesystem presents data as a hierarchy of named objects. This part examined directory contents, basic file operations, ownership, and access permissions.

1.1 Directory structure

The home directory contains personal files and application settings. The `/etc` listing includes configuration entries such as `NetworkManager`, `X11`, and `adduser.conf`. Figure 1.1 shows these listings.

Two directories directly under the root are `/etc`, which contains system configuration, and `/home`, which contains users' home directories. The home and `/etc` listings show personal files and system configuration entries respectively.

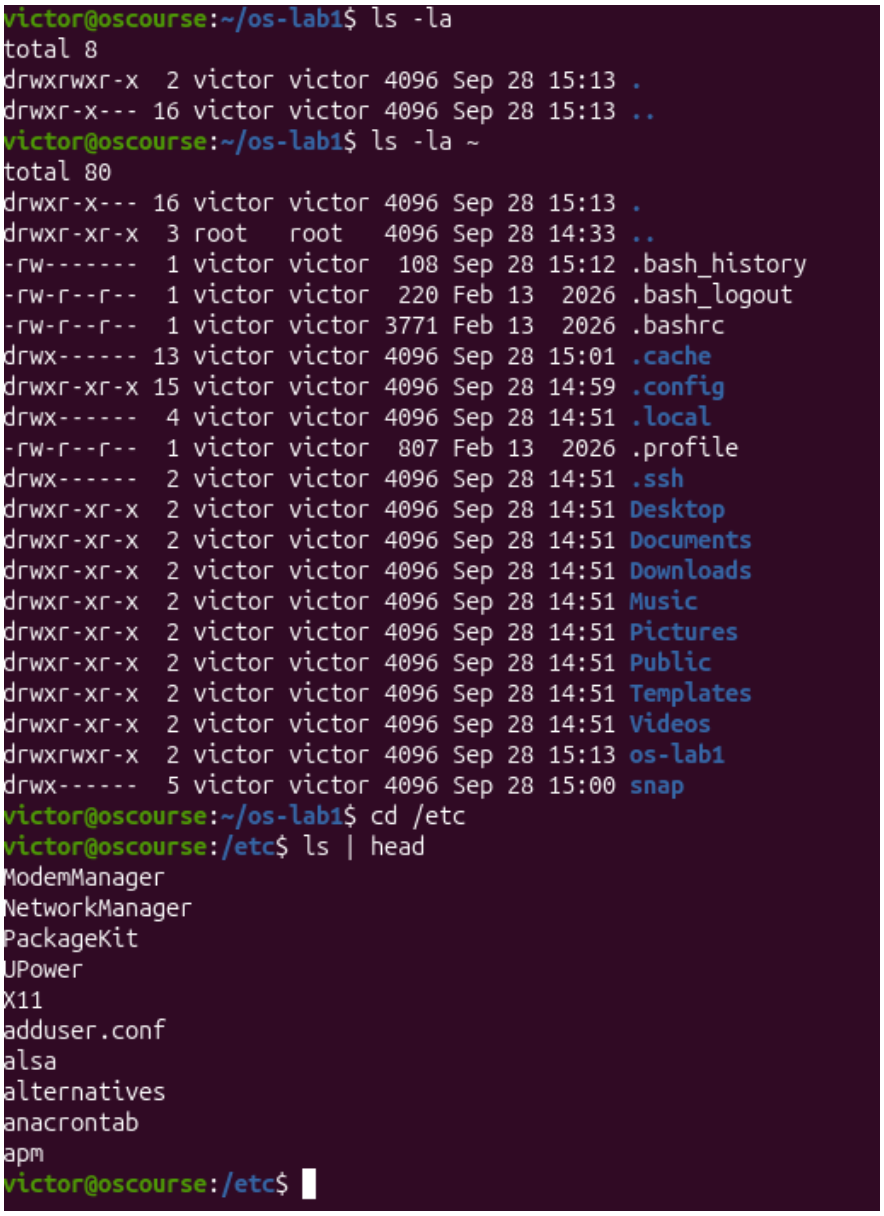


Figure 1.1 – Working, home, and configuration directory listings

The root-directory commands were run on the openSUSE Tumbleweed host. The working directory is `/home/victor/Projects/os-labs/lab1`. The listing shows the top-level directories and their ownership and modes; `/bin` is a symbolic link to `usr/bin` on this host.

```
$ date -Is
2026-10-26T11:47:16+02:00
$ grep '^PRETTY_NAME=' /etc/os-release
PRETTY_NAME="openSUSE Tumbleweed"
$ uname -sr
Linux 7.2.7-1-default
$ pwd
/home/victor/Projects/os-labs/lab1
$ ls -la /
total 16
drwxr-xr-x.   1 root root  142 Sep  1 16:21 .
drwxr-xr-x.   1 root root  142 Sep  1 16:21 ..
drwxr-x---.   1 root root   66 Oct 11 17:59 .snapshots
lrwxrwxrwx.   1 root root    7 Mar  2  2026 bin -> usr/bin
dr-xr-xr-x.   1 root root  356 Sep 28 17:10 boot
drwxr-xr-x.  20 root root 4400 Oct 23 13:50 dev
drwxr-xr-x.   1 root root 3654 Oct 26 11:31 etc
dr-xr-xr-x.   1 root root   12 Sep  1 16:27 home
lrwxrwxrwx.   1 root root    7 Mar  2  2026 lib -> usr/lib
lrwxrwxrwx.   1 root root    9 Mar  2  2026 lib64 -> usr/lib64
dr-xr-xr-x.   1 root root    0 Sep  1 16:21 mnt
dr-xr-xr-x.   1 root root   78 Sep 29 11:12 opt
dr-xr-xr-x. 340 root root    0 Oct  6 13:15 proc
drwx------.   1 root root  106 Sep 17 11:12 root
drwxr-xr-x.  52 root root 1280 Oct  6 10:16 run
lrwxrwxrwx.   1 root root    8 Mar  2  2026 sbin -> usr/sbin
dr-xr-xr-x.   1 root root   22 Sep  1 16:52 srv
dr-xr-xr-x.  13 root root    0 Oct 26 11:46 sys
drwxrwxrwt.  36 root root 1200 Oct 26 11:47 tmp
drwxr-xr-x.   1 root root  124 Sep  1 16:21 usr
drwxr-xr-x.   1 root root  120 Sep  1 16:52 var
```

Figure 1.2 – Working directory and root hierarchy on the openSUSE host

1.2 Creation, copying, renaming, and removal

The command `echo` created `note.txt` containing `hello operating systems`. Copying it produced `copy.txt`, which was renamed to `renamed.txt`. Both files were 24 bytes long. After `rm renamed.txt`, the directory contained only `note.txt`. Figure 1.3 documents the commands and the before-and-after listings. The owner and group of both created files were `victor`.

```
victor@oscourse:/etc$ cd ~/os-lab1
victor@oscourse:~/os-lab1$ mkdir -p demo
victor@oscourse:~/os-lab1$ cd demo
victor@oscourse:~/os-lab1/demo$ echo "hello operating systems" > note.txt
victor@oscourse:~/os-lab1/demo$ cp note.txt copy.txt
victor@oscourse:~/os-lab1/demo$ mv copy.txt renamed.txt
victor@oscourse:~/os-lab1/demo$ ls -l
total 8
-rw-rw-r-- 1 victor victor 24 Sep 28 15:20 note.txt
-rw-rw-r-- 1 victor victor 24 Sep 28 15:20 renamed.txt
victor@oscourse:~/os-lab1/demo$ cat note.txt
hello operating systems
victor@oscourse:~/os-lab1/demo$ rm renamed.txt
victor@oscourse:~/os-lab1/demo$ ls -l
total 4
-rw-rw-r-- 1 victor victor 24 Sep 28 15:20 note.txt
victor@oscourse:~/os-lab1/demo$
```

Figure 1.3 – Creating, copying, renaming, and deleting a file

1.3 Ownership and permissions

The initial mode was `-rw-rw-r--`. After `chmod 600 note.txt`, it became `-rw-------`. In this ten-character field, the first character identifies a regular file. The next three characters give the owner read and write permission without execute permission. The remaining groups of three give the group and other users no access. The command `chmod 644 note.txt` restored `-rw-r--r--`, allowing other users to read while retaining write permission only for the owner. Figure 1.4 shows all three modes.

```
victor@oscourse:~/os-lab1/demo$ ls -l
total 4
-rw-rw-r-- 1 victor victor 24 Sep 28 15:20 note.txt
victor@oscourse:~/os-lab1/demo$ ls -l note.txt
-rw-rw-r-- 1 victor victor 24 Sep 28 15:20 note.txt
victor@oscourse:~/os-lab1/demo$ chmod 600 note.txt
victor@oscourse:~/os-lab1/demo$ ls -l note.txt
-rw------- 1 victor victor 24 Sep 28 15:20 note.txt
victor@oscourse:~/os-lab1/demo$ chmod 644 note.txt
victor@oscourse:~/os-lab1/demo$ ls -l note.txt
-rw-r--r-- 1 victor victor 24 Sep 28 15:20 note.txt
victor@oscourse:~/os-lab1/demo$
```

Figure 1.4 – Permission changes from the initial mode to 600 and 644

2 PROCESS MANAGEMENT

A process is an executing program with an identifier and execution state. Process-listing tools and the kernel's `/proc` interface expose the tasks being managed by the OS.

2.1 Listing processes and identifying PID 1

The initial `ps aux | wc -l` command printed 223, including the output header. The header-free command `ps -e --no-headers | wc -l` printed 222. This indicates approximately 222 processes at that snapshot; counts can change even between consecutive commands.

Process 1 was `systemd`, running as root from `/usr/lib/systemd/systemd`. It is the initial userspace process and manages system services. Figure 2.1 records its identity and the process counts.

```
victor@oscourse:~/os-lab1/demo$ ps aux | head
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.5  0.4  25380 16684 ?        Ss   15:07   0:05 /usr/lib/systemd/systemd --switched-root --system --deserialize=51
root           2  0.0  0.0      0     0 ?        S    15:07   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    15:07   0:00 [pool_workqueue_release]
root           4  0.0  0.0      0     0 ?        I<   15:07   0:00 [kworker/R-rcu_gp]
root           5  0.0  0.0      0     0 ?        I<   15:07   0:00 [kworker/R-sync_wq]
root           6  0.0  0.0      0     0 ?        I<   15:07   0:00 [kworker/R-kvfree_rcu_reclaim]
root           7  0.0  0.0      0     0 ?        I<   15:07   0:00 [kworker/R-slub_flushwq]
root           8  0.0  0.0      0     0 ?        I<   15:07   0:00 [kworker/R-netns]
root          10  0.0  0.0      0     0 ?        I<   15:07   0:00 [kworker/0:0H]
victor@oscourse:~/os-lab1/demo$ ps aux | wc -l
223
victor@oscourse:~/os-lab1/demo$ ps -e --no-headers | wc -l
222
victor@oscourse:~/os-lab1/demo$ ps -p 1 -o pid,comm,args
    PID COMMAND         COMMAND
      1 systemd         /usr/lib/systemd/systemd --switched-root --system --deserialize=51
victor@oscourse:~/os-lab1/demo$
```

Figure 2.1 – Process listing, approximate count, and identity of PID 1

The live monitor in Figure 2.2 showed 251 tasks: one running and 250 sleeping, with no stopped or zombie tasks. This later snapshot differs from the earlier `ps` count because the VM's workload changed.

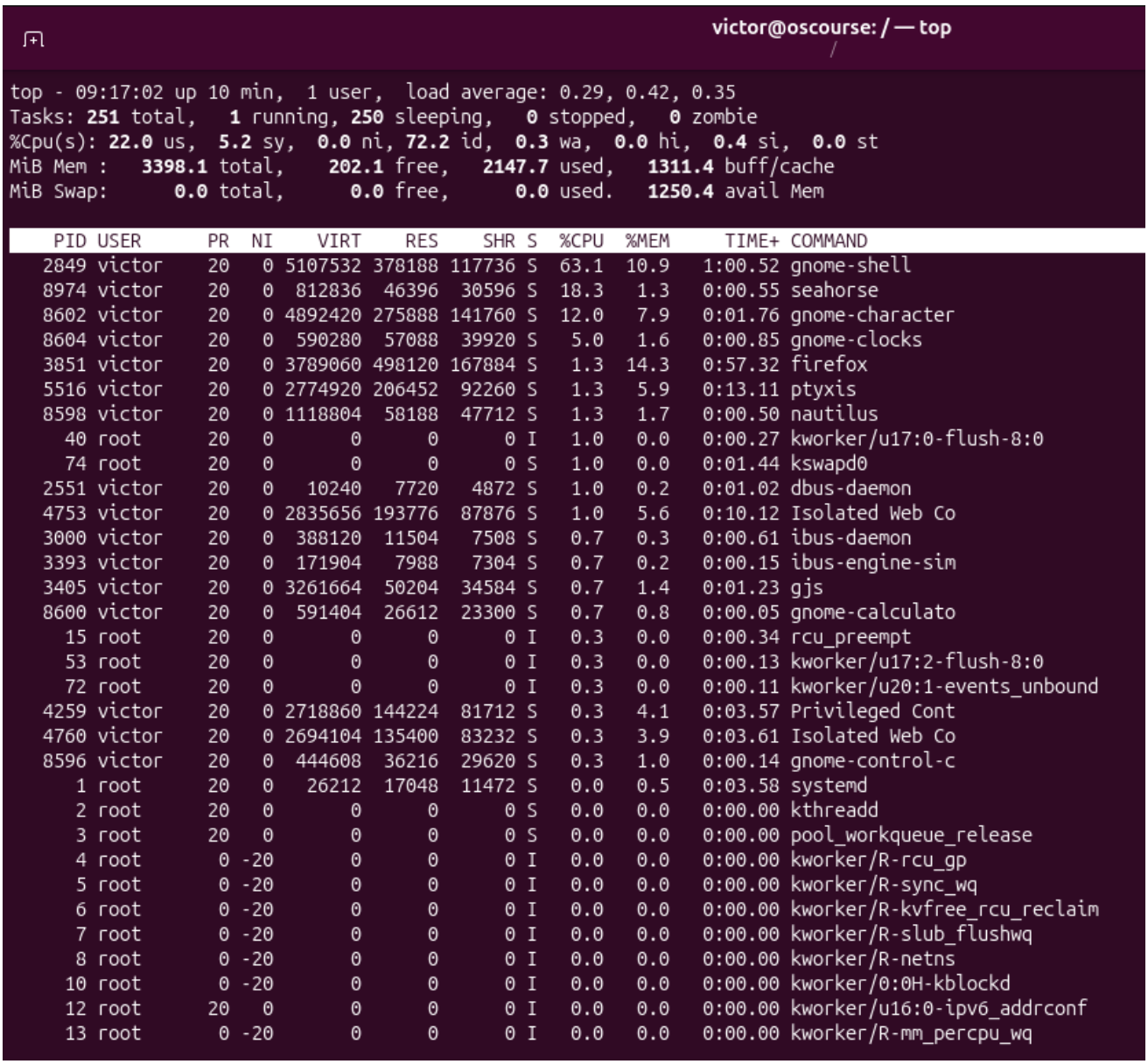


Figure 2.2 – Live CPU, memory, and process information from top

2.2 Starting and terminating a background task

The command `sleep 300 &` started a background process with PID 9453. The shell's `jobs` command and the system-wide `ps` listing both showed it running. After `kill "$PID"`, the shell reported termination. The final process lookup printed only a header, indicating that the process was no longer present. Figure 2.3 records this lifecycle.

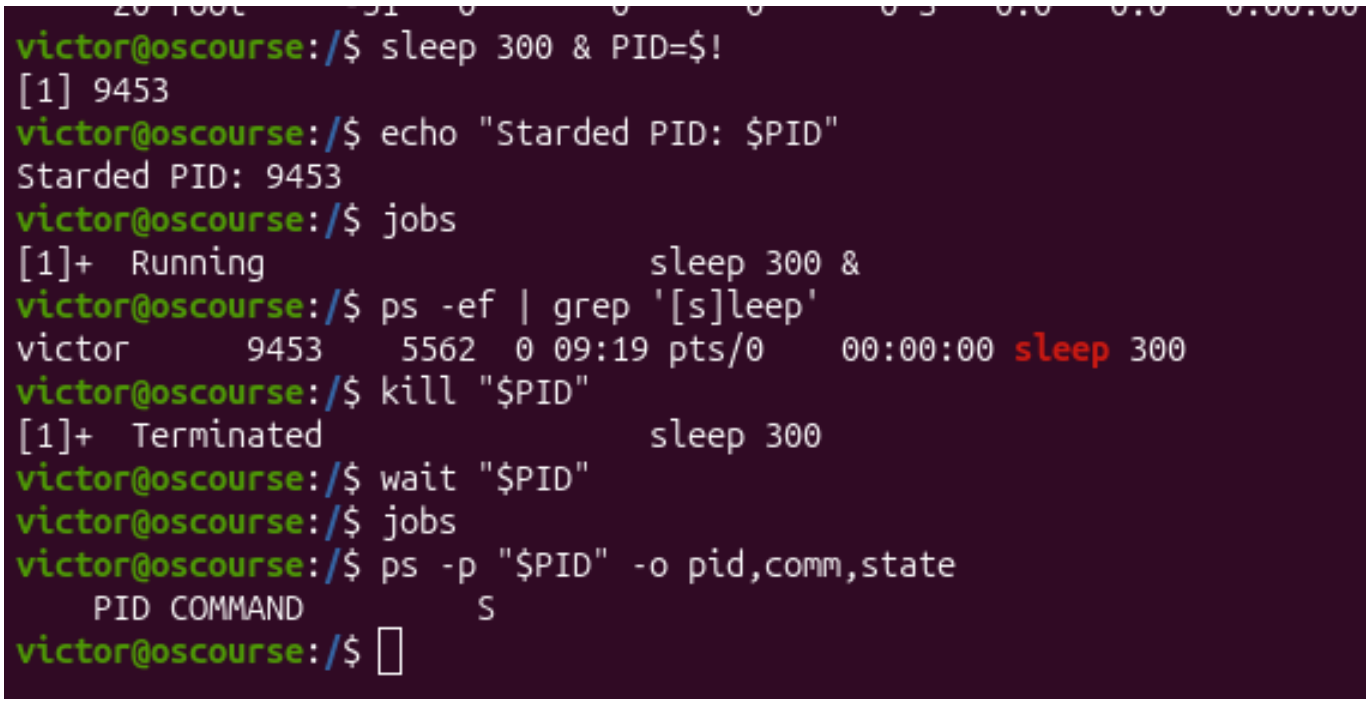


Figure 2.3 – Starting, finding, and terminating a background process

2.3 Inspecting process state through the kernel

A separate `sleep` process had PID 9765. Its directory under `/proc` contained entries such as `cmdline`, `fd`, `maps`, and `status`. The status file reported `State: S (sleeping)`. This is interruptible sleep: the process waits rather than actively executing on the CPU. Figure 2.4 also shows the process and parent identifiers and its termination after inspection.

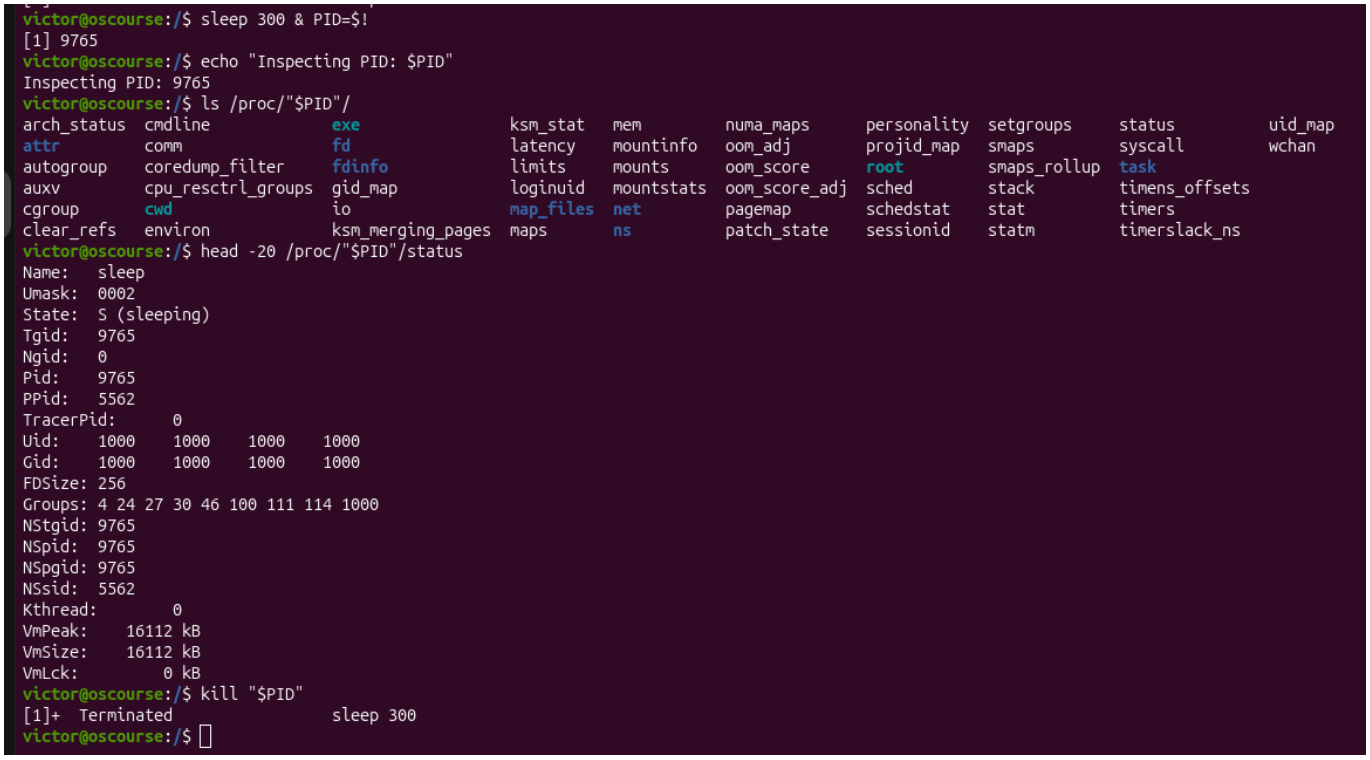


Figure 2.4 – Kernel-exposed status of the sleeping process

3 MEMORY MANAGEMENT

The OS distributes physical memory between processes and uses reclaimable cache to improve performance. System totals and per-process memory fields provide different views of this allocation.

3.1 Physical memory and swap

Table 3.1 summarizes the `free -h` snapshot.

Table 3.1 – Observed memory totals

Field	Reported value
Total RAM	3.3 GiB
Used RAM	2.4 GiB
Free RAM	272 MiB
Available RAM	962 MiB
Configured swap	0 B

Free memory is unused memory, whereas available memory also accounts for memory that can be reclaimed for applications. Swap is backing space used to move eligible memory pages out of RAM; none was configured in this capture. The subsequent `/proc/meminfo` reading reported `MemTotal: 3479664 kB`. The commands were separate snapshots, so their free and available values need not match exactly.

3.2 Resident memory of a sleeping process

The process with PID 9978 reported `VmRSS: 7752 kB`, approximately 7.57 MiB. This is more than the source-level simplicity of `sleep` alone suggests, but an executing program still requires code, stack, runtime data, and possibly shared library pages. RSS counts resident shared pages as well as private pages and therefore does not measure exclusively owned memory. Figure 3.1 records both system and process readings.



Figure 3.1 – System memory, swap, and resident memory of sleep

4 DEVICE AND STORAGE MANAGEMENT

Storage tools connect logical mount points with the devices that provide their data. The device directory also exposes file-like interfaces for I/O.

4.1 Root filesystem and disk usage

The root filesystem / was mounted from /dev/sda2, using ext4, as confirmed by findmnt /. The df -h output reported 49 GiB total, 7.5 GiB used, 39 GiB available, and 17% utilization. The block-device listing showed a 50 GiB virtual disk named sda. The working folder occupied 12 KiB according to du -sh. Figure 4.1 connects the disk, partition, and mount point.

```
victor@oscourse:/$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           680M  2.7M  677M   1% /run
/dev/sda2        49G  7.5G   39G  17% /
tmpfs           1.7G     0  1.7G   0% /dev/shm
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs           1.7G  7.4M  1.7G   1% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
tmpfs           340M   92K  340M   1% /run/user/1000
/dev/sr0         51M   51M     0 100% /run/media/victor/VBox_GAs_7.2.18
victor@oscourse:/$ lsblk
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0    7:0    0     4K  1 loop /snap/bare/5
loop1    7:1    0  66.8M  1 loop /snap/core24/1643
loop2    7:2    0    20M  1 loop /snap/desktop-security-center/151
loop3    7:3    0 260.3M  1 loop /snap/firefox/8763
loop4    7:4    0  16.5M  1 loop /snap/firmware-updater/226
loop5    7:5    0 614.5M  1 loop /snap/gnome-46-2404/164
loop6    7:6    0  91.7M  1 loop /snap/gtk-common-themes/1535
loop7    7:7    0   1.5M  1 loop /snap/hwctl/123
loop8    7:8    0   402M  1 loop /snap/mesa-2404/1839
loop9    7:9    0  18.8M  1 loop /snap/prompting-client/222
loop10   7:10   0  11.8M  1 loop /snap/snap-store/1390
loop11   7:11   0  50.1M  1 loop /snap/snapd/27710
loop12   7:12   0   828K  1 loop /snap/snapd-desktop-integration/391
loop13   7:13   0  14.2M  1 loop /snap/desktop-security-center/188
loop14   7:14   0  18.4M  1 loop /snap/prompting-client/228
sda      8:0    0    50G  0 disk
├─sda1   8:1    0     1M  0 part
└─sda2   8:2    0    50G  0 part /
sr0     11:0    1  50.7M  1 rom  /run/media/victor/VBox_GAs_7.2.18
victor@oscourse:/$ du -sh ~/os-lab1
12K     /home/victor/os-lab1
victor@oscourse:/$ findmnt /
TARGET SOURCE    FSTYPE OPTIONS
/      /dev/sda2 ext4   rw,relatime
victor@oscourse:/$
```

Figure 4.1 – Mounted filesystems, block devices, and working-folder size

4.2 Device entries and file-like interfaces

The entry `/dev/cdrom` points to `sr0`, the VM's virtual optical drive. The storage listing shows VirtualBox Guest Additions media mounted from `/dev/sr0`. The entry `/dev/null` is a character device that discards written data and returns end-of-file when read.

The UNIX expression "everything is a file" describes how many resources, including devices, are exposed through operations such as opening, reading, and writing. It does not mean every resource is a regular disk file. Figure 4.2 shows device types, symbolic links, and mounts.

```
victor@oscourse:/$ ls -l /dev | head
total 0
crw-r--r--  1 root   root     10, 235 Oct  1 09:06 autofs
drwxr-xr-x  2 root   root         420 Oct  1 09:11 block
drwxr-xr-x  2 root   root          80 Oct  1 09:06 bsg
crw-------  1 root   root     10, 234 Oct  1 09:06 btrfs-control
drwxr-xr-x  3 root   root          60 Oct  1 09:06 bus
lrwxrwxrwx+ 1 root   root           3 Oct  1 09:06 cdrom -> sr0
drwxr-xr-x  2 root   root        3780 Oct  1 09:07 char
crw-------  1 root   tty       5,   1 Oct  1 09:06 console
lrwxrwxrwx  1 root   root          11 Oct  1 09:06 core -> /proc/kcore
victor@oscourse:/$ ls -l /dev/null
crw-rw-rw- 1 root root 1, 3 Oct  1 09:06 /dev/null
victor@oscourse:/$ mount | head
tmpfs on /run type tmpfs (rw,nosuid,nodev,size=695936k,nr_inodes=819200,mode=755,inode64)
/dev/sda2 on / type ext4 (rw,relatime)
devtmpfs on /dev type devtmpfs (rw,nosuid,size=779156k,nr_inodes=194789,mode=755,inode64)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64,usrquota)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=600,ptmxmode=000)
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime,nsdelegate,memory_recursiveprot,memory_hugetlb_accounting)
none on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime)
bpf on /sys/fs/bpf type bpf (rw,nosuid,nodev,noexec,relatime,mode=700)
victor@oscourse:/$
```

Figure 4.2 – Device entries and filesystem mount interfaces

CONCLUSIONS

The OS manages files and directories, whose ownership and permissions were inspected with `ls -l`. It manages processes, visible with `ps`, and memory, visible with `free -h`. It also manages devices and storage, which were inspected with `lsblk`.

The observations demonstrate permission enforcement, process termination, kernel-exposed process state, and the association between a mounted filesystem and its device. Process counts and memory use varied between captures, as expected for a running system. All reported measurements are recorded in the terminal screenshots and command transcript.